AlertMind

An AI-Assisted Mini Security Operations Centre

Does an LLM tier-1 assistant actually improve alert triage — and where does it fail?

Capstone · CAP-SCE-3W · SOC Essentials

PG Certificate in AI/GenAI Powered Cybersecurity — IIT Roorkee × Futurense

THE PROBLEM

A three-person SOC, drowning in alerts

~1,200	35 min	3
alerts / day	mean time-to-triage	analysts

This project is the in-house pilot crew — build the SOC, add the assistant, and measure whether it helps.

Solo build · Windows + Linux endpoints · isolated VirtualBox lab

Leadership wants to

1. **Consolidate**
   one SIEM, heterogeneous log sources
2. **Cut alert fatigue**
   fewer false positives to chase
3. **Prototype an LLM assistant**
   before buying a commercial SOAR

THE BUILD

What is built — and what is not

Implemented — the SOC platform

Wazuh 4.14	manager · indexer · dashboard	**Telemetry**	Windows Sysmon · Linux auditd
24 rules	ATT&CK-mapped, all verified firing	**2 dashboards · 3 playbooks**	SOC briefing · heatmap · NIST 800-61r2 lifecycle

Implemented — what the assistant actually reads

The frozen 20-alert corpus · analyst-pasted JSON via the Paste & inspect tab

Planned — documented target state, NOT implemented

Live read-only Wazuh API ingestion · RBAC identities socanalyst and assistant-svc

DETECTION ENGINEERING

Rules mapped to ATT&CK — and honestly tuned

24	7+17	5	100%
custom rules	Windows / Linux	ATT&CK tactics	verified firing

Tuning discipline, not just coverage

- LSASS-access rule took three rounds to separate credential dumps from the security tooling's own benign reads (~557 → 0 false positives).
- Sigma authored as the portable artifact, then hand-translated to Wazuh where the pySigma backend fell short — every translation logged.
- Known residual false positives are documented, not hidden (e.g. cron reads of /etc/shadow) — the same hard cases the assistant is later tested on.

THE LLM ASSISTANT

Four triage outputs, behind a strict pipeline

- 5-line summary
- ATT&CK technique tag
- Investigation queries
- Draft user message

Every alert runs the same path

1. **Redact secrets (before any prompt)**
2. Apply view
3. Build prompt (untrusted-data block)
4. Call LLM
5. Parse + validate schema
6. Log (25 fields) · Score

Redaction runs first — tested secret values are removed before the prompt is constructed.

SAFETY CONTROLS

Controls enforced in code, with evidence

Tested secrets redacted before the model

redaction runs before prompt construction

- **0 / 7 planted values leaked**

No autonomous action

no tools, no send path; text-only draft

- **analyst review required on every output**

Injection visibility and containment

markers flagged in keys and values

- **delimiter attempts blocked pre-call**

Reconstructable batch audit

per-run directory, never overwritten

- **25-field records; Paste audit saved explicitly**

- **Live demo —** the **Paste & inspect** tab runs a synthetic alert through this same pipeline: watch secrets get masked and an injection payload flagged, in real time.

HOW WE MEASURED

Two moves that make the evaluation honest

1 Salt the corpus

20 alerts = 14 controlled attack alerts + 6 real false positives from the tuning history.

Without false positives, an assistant that confirms everything scores 100% — and the measurement is worthless.

2 Strip the answer

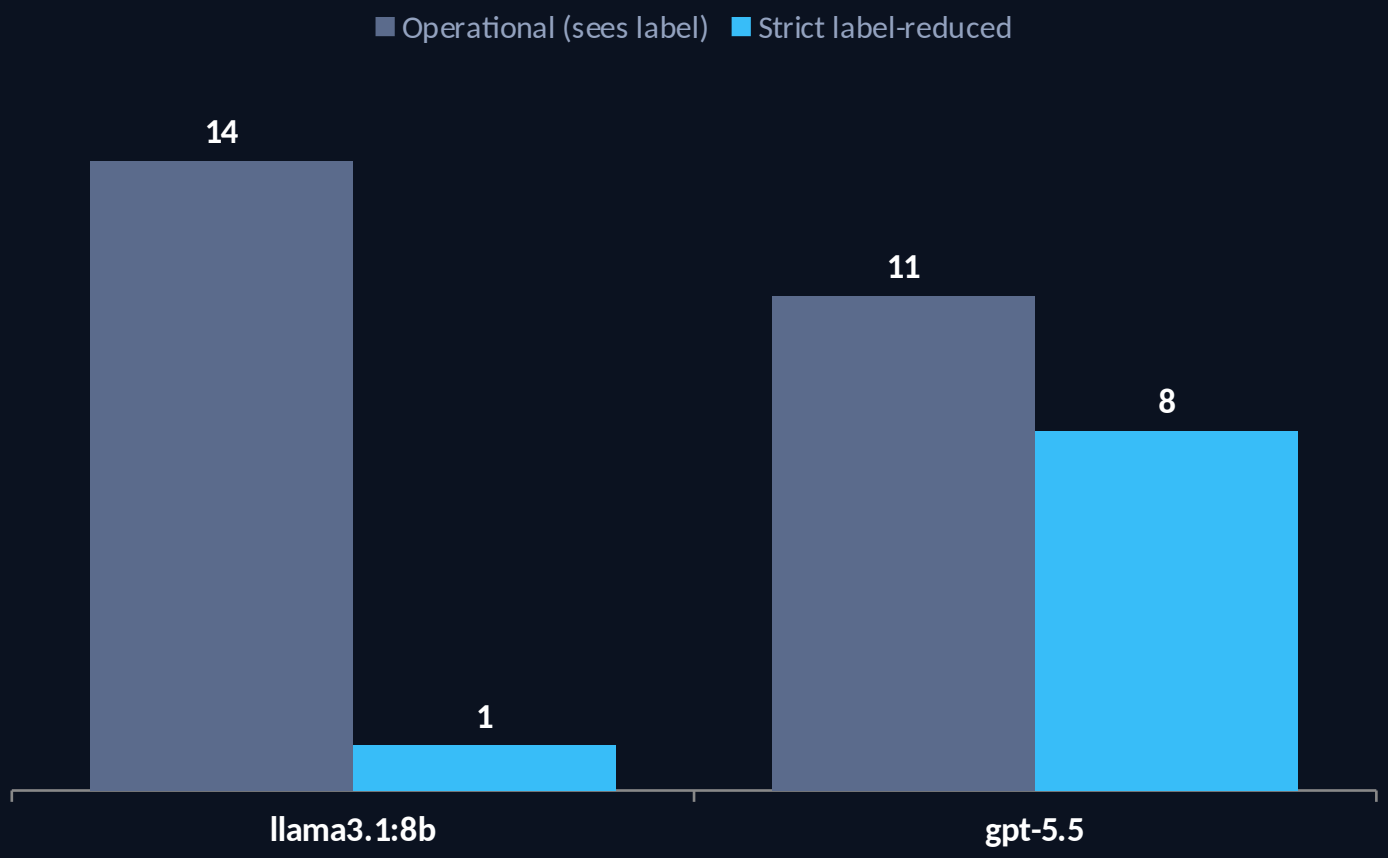
A strict label-reduced view removes the rule's own ATT&CK label, description and detection keys.

Otherwise "right technique?" only tests whether the model can copy the label — not classify.

Scoring separates technique · disposition · consistency — a right tag with a contradictory call cannot score correct.

FINDING 1 · LABEL LEAKAGE

Strip the label, and one model collapses



ATT&CK technique accuracy — attacks (exact-ID overlap, of 14)

14 → 1

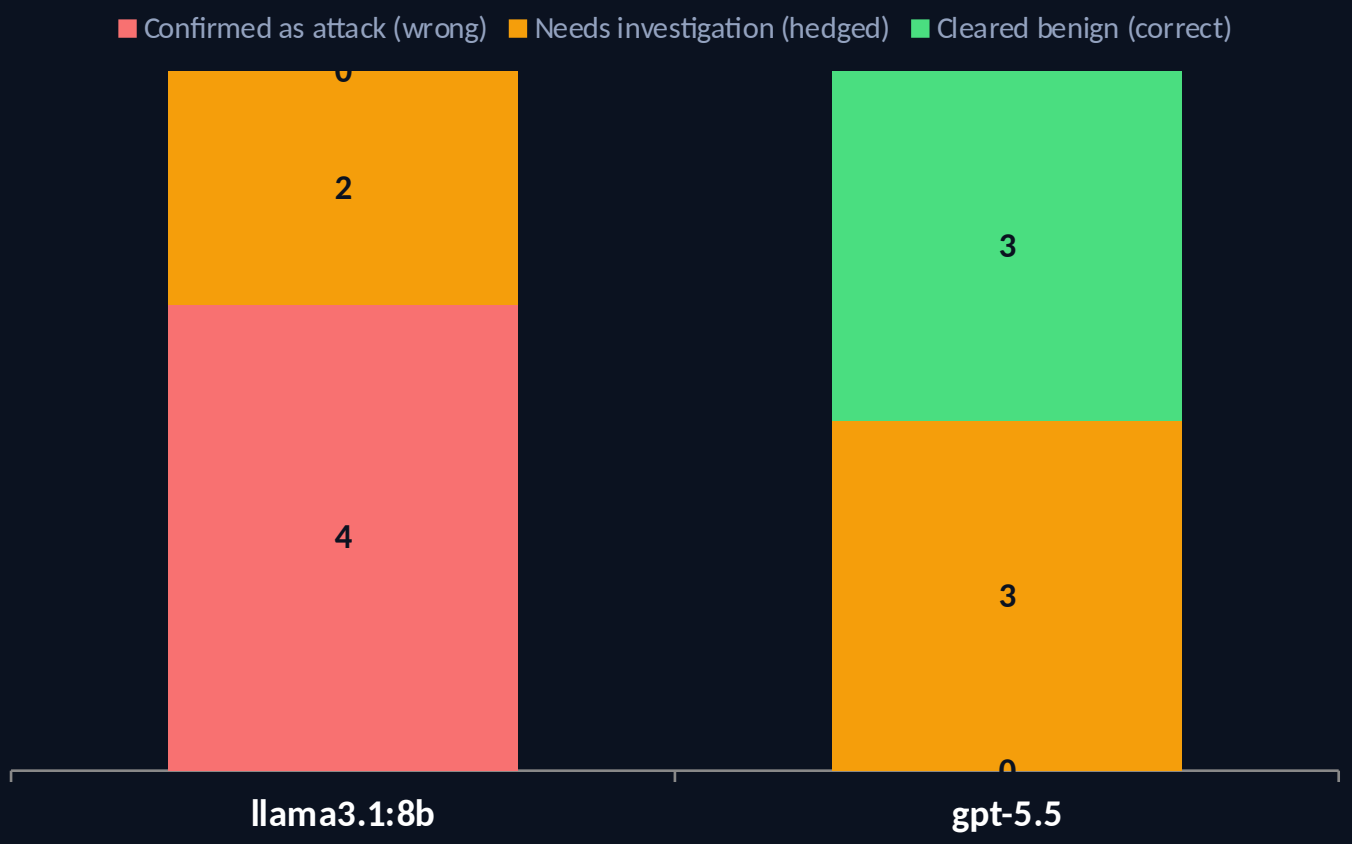
Consistent with heavy reliance on the exposed rule metadata.

11 → 8

GPT-5.5 retained more exact credit after metadata removal.

FINDING 2 · DISPOSITION

The weak model never says "benign"



Disposition on the 6 benign false positives (strict view)

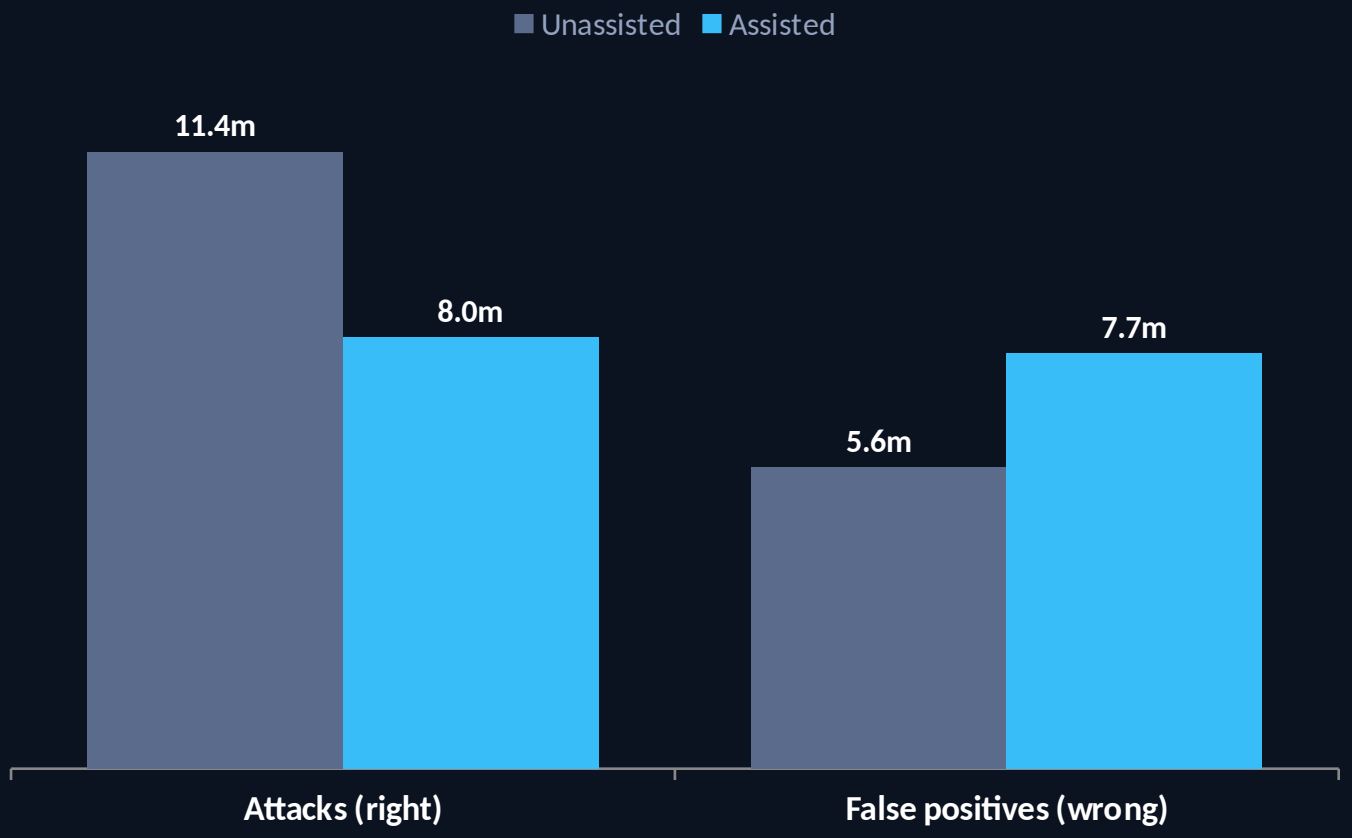
0 / 6

llama3.1 cleared none: four confidently misclassified as attacks, two referred for investigation — including the security tools' own LSASS reads.

This inverts the goal: false positives ARE the alert-fatigue workload the pilot set out to cut.

FINDING 3 · THE HEADLINE

Assisted triage: faster on attacks, slower on false positives



−30% on the 14 it got right

+38% on the 6 it got wrong

Faster on 14/14, slower on 6/6 — no exceptions.

20 / 20

analyst accuracy in both conditions — the human overrode all 6 AI errors. Paired false-positive penalty: median +1.68 min (range +0.97 to +2.47).

Median triage time, minutes. In this corpus alert class and assistant correctness coincide, so the association is categorical but the causal mechanism is not isolated.

FINDING 4 · ARE THE WORDS TRUSTWORTHY?

Grounding the free text, not just the tag

Manual grounding review — all six dimensions, single reviewer, operational outputs (n = 20 per model)

	llama3.1:8b	gpt-5.5
Summary fully supported	15/20	20/20
Unsupported statements	5	0
Investigation queries runnable	0/20	20/20
Investigation queries relevant	20/20	20/20
Draft message appropriate	19/20	20/20
Confidence judged appropriate to evidence	13/20	20/20

llama3.1's queries were relevant but 0/20 runnable — vague prose an analyst cannot execute. Five summaries carried an unsupported claim, including one factual error.

THREATS TO VALIDITY

What this pilot does not prove

- **Small n, single environment**

  20 alerts, one analyst — directional, not statistically powered

- **Self-generation bias**

  the analyst built the attacks and knew the answers

- **Exploratory comparison**

  model, hosting & inference config all changed together; gpt-5.5 is one stochastic sample per view

- **Learning effect**

  assisted pass re-saw the corpus; washout + randomised order bound it, but no counterbalanced crossover

- **Construct validity (A18)**

  one "attack" announced itself as a test — gpt-5.5 was right about the world, scored wrong

- **Redaction is risk-reduction**

  tested credential classes only, not a guarantee

CONCLUSION

The deployment gate

Not *"is it fast?"* → **"is it right on the alerts you'd otherwise dismiss?"**

llama3.1:8b

Do not deploy for false-positive triage — 0/6, and it slows that work down.

gpt-5.5

Promising candidate for further evaluation — not deployment-approved on one run.

The method

Benign salt + label-free view + grounding is what made any of this checkable.

An LLM assistant's value is conditional on its correctness — and its failures are not neutral, they are actively costly.

Thank you

Questions & defense

Working SIEM + 24 verified rules · guardrailed assistant · measured impact

Repository: github.com/opandey1/alertmind